

Automation Landscape

Series: AUTOM | **Notebook:** 1 of 8 | **Created:** January 2026 | **Last Updated:** 01/28/2026

Table of Contents

- 1. [Introduction](#)
- 2. [Automation Options Overview](#)
- 3. [Choosing the Right Tool](#)
- 4. [Decision Framework](#)
- 5. [Next Steps](#)

Prerequisites

Before starting this series, ensure you have:

Requirement	Description
Dynatrace SaaS tenant	Active tenant with admin access
API Token	Token with appropriate scopes for automation
Basic familiarity	Understanding of Dynatrace configuration concepts

Learning Objectives

By the end of this notebook, you will:

- Understand all available automation options for Dynatrace
 - Know when to use each tool or approach
 - Be able to choose the right automation strategy for your use case
 - Have a framework for evaluating automation approaches
-

1. Introduction

Dynatrace provides multiple ways to automate configuration management and operational tasks. This series covers all major automation options, helping you choose the right approach for your needs.

What This Series Covers

Notebook	Focus
AUTOM-01 (this notebook)	Automation landscape overview
AUTOM-02	Settings API - REST API for configuration
AUTOM-03	Monaco - Configuration-as-code CLI
AUTOM-04	Terraform Provider - Infrastructure-as-code
AUTOM-05	Dynatrace Workflows - Event-driven automation
AUTOM-06	Dynatrace SDKs - TypeScript and Python clients
AUTOM-07	CI/CD Integration - GitOps patterns
AUTOM-08	Migration Automation - Bulk configuration transfer

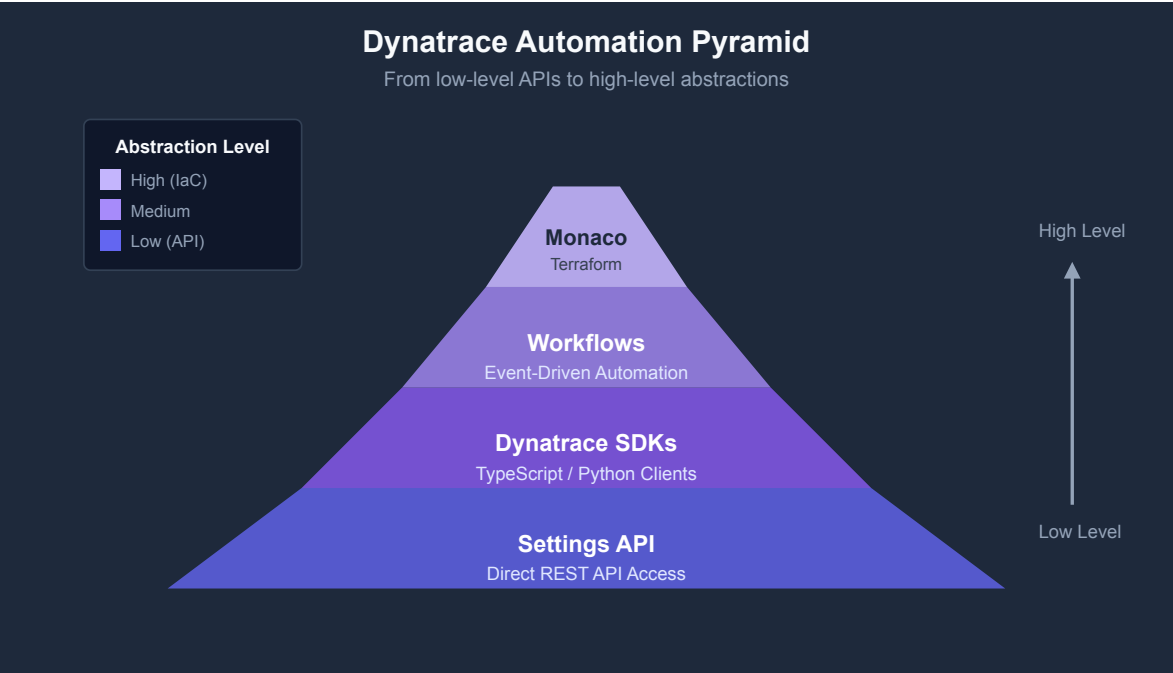
Why Automate?

Benefit	Description
Consistency	Same configuration across all environments
Repeatability	Deploy identical setups reliably
Version Control	Track changes in Git, enable rollbacks
Scale	Manage hundreds or thousands of configurations
Speed	Faster deployments, reduced manual effort
Compliance	Auditable changes, approval workflows

2. Automation Options Overview

The Automation Pyramid

Dynatrace automation tools form a hierarchy from low-level APIs to high-level abstractions:



Tool Comparison Matrix

Tool	Best For	Learning Curve	CI/CD Ready	Multi-Tenant
Settings API	Custom integrations, scripts	Medium	Manual	Yes
Monaco	Config-as-code, migrations	Low	Yes	Yes
Terraform	IaC environments, full stack	Medium	Yes	Yes
Workflows	Event-driven, auto-remediation	Low	N/A	No
SDKs	Custom apps, complex logic	Medium	Yes	Yes

Settings API

The foundation for all configuration automation. Direct REST API access to Dynatrace settings.

Aspect	Details
Type	REST API
Access	HTTP calls with API token
Schema	JSON with schema validation
Use Case	Custom scripts, one-off operations
Documentation	Settings API Reference

Monaco

Dynatrace's official configuration-as-code CLI tool. YAML-based configuration management.

Aspect	Details
Type	CLI tool
Configuration	YAML files
Features	Download, deploy, delete, validate
Use Case	Config management, migrations
Documentation	Monaco on GitHub

Terraform Provider

Official HashiCorp Terraform provider for Dynatrace. Infrastructure-as-code approach.

Aspect	Details
Type	Terraform provider
Configuration	HCL (HashiCorp Configuration Language)
Features	Plan, apply, destroy, state management
Use Case	IaC pipelines, environment provisioning
Documentation	Terraform Registry

Dynatrace Workflows

Built-in automation engine for event-driven actions within Dynatrace.

Aspect	Details
Type	Platform feature
Configuration	UI or YAML
Features	Triggers, actions, conditions, JavaScript
Use Case	Auto-remediation, notifications, integrations
Documentation	Workflows Documentation

Dynatrace SDKs

Official client libraries for programmatic access to Dynatrace APIs.

Aspect	Details
Type	Client libraries
Languages	TypeScript/JavaScript, Python
Features	Type-safe, auto-generated from OpenAPI
Use Case	Custom applications, complex automation
Documentation	Dynatrace SDK

3. Choosing the Right Tool

Use Case Mapping

Use Case	Recommended Tool	Why
One-off configuration change	Settings API	Simple, direct
Repeatable deployments	Monaco	Version-controlled YAML
Full environment provisioning	Terraform	State management, drift detection
Auto-remediation	Workflows	Event-driven, built-in
Custom application	SDK	Type-safe, maintainable
Tenant migration	Monaco	Download/deploy pattern
GitOps pipeline	Monaco or Terraform	CI/CD integration

Team Skill Considerations

Team Background	Best Fit
DevOps with Terraform experience	Terraform Provider
Developers comfortable with APIs	Settings API or SDK
SRE teams wanting GitOps	Monaco
Operations needing quick automation	Workflows
Mixed skill levels	Monaco (lowest barrier)

Feature Coverage Comparison

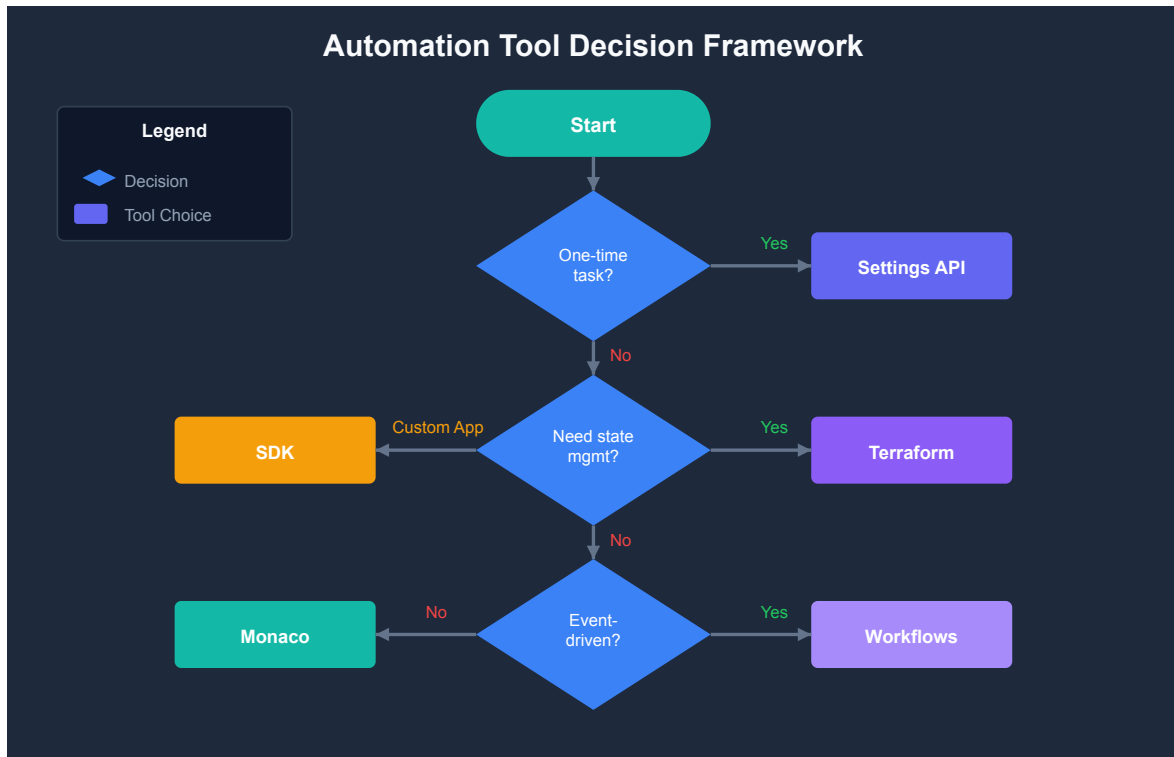
Not all tools support all Dynatrace features equally:

Configuration Type	Settings API	Monaco	Terraform
Settings 2.0 objects	Full	Full	Full
Classic config (legacy)	N/A	Full	Full
Dashboards	Full	Full	Full
Synthetic monitors	Full	Full	Full
Alerting profiles	Full	Full	Full
Management zones	Full	Full	Full
Auto-tagging rules	Full	Full	Full
SLOs	Full	Full	Full
Workflows	Full	Partial	Partial
OpenPipeline	Full	Full	Full

Note: Monaco and Terraform use the Settings API under the hood. Feature parity depends on schema availability.

4. Decision Framework

Use this flowchart to choose the right automation approach:



Combining Tools

These tools aren't mutually exclusive. Common combinations:

Combination	Use Case
Monaco + Workflows	Deploy configs with auto-remediation
Terraform + Monaco	Infra provisioning + config management
SDK + Settings API	Custom app with direct API fallback
Monaco + CI/CD	GitOps config deployments

Anti-Patterns to Avoid

Anti-Pattern	Problem	Solution
Mixing Monaco and Terraform for same config	State conflicts	Choose one for each config type
Manual changes alongside automation	Configuration drift	Use automation exclusively
No version control	No rollback capability	Always use Git
Hardcoded secrets	Security risk	Use environment variables or vaults

API Token Scopes Reference

Each automation tool requires specific API token scopes:

Tool	Required Scopes
Settings API	<code>settings.read</code> , <code>settings.write</code>
Monaco	<code>settings.read</code> , <code>settings.write</code> , <code>ReadConfig</code> , <code>WriteConfig</code>
Terraform	<code>settings.read</code> , <code>settings.write</code> , <code>ReadConfig</code> , <code>WriteConfig</code>
Workflows	Built-in (no external token needed)
SDKs	Depends on operations performed

Token Best Practices

Practice	Description
Least privilege	Only grant required scopes
Environment-specific	Separate tokens per environment
Rotation	Rotate tokens regularly
Secret storage	Use HashiCorp Vault, AWS Secrets Manager, etc.

5. Next Steps

Learning Path by Goal

Your Goal	Recommended Path
Quick script automation	AUTOM-02 (Settings API)
GitOps config management	AUTOM-03 (Monaco) → AUTOM-07 (CI/CD)
Full IaC environment	AUTOM-04 (Terraform) → AUTOM-07 (CI/CD)
Auto-remediation	AUTOM-05 (Workflows)
Custom application	AUTOM-06 (SDKs)
Tenant migration	AUTOM-08 (Migration)

Continue the Series

Next Notebook	Focus
AUTOM-02: Settings API	Deep dive into REST API configuration

Additional Resources

- [Dynatrace API Documentation](#)
- [Monaco GitHub Repository](#)

- [Terraform Provider Documentation](#)
 - [Dynatrace Developer Portal](#)
-

Summary

In this notebook, you learned:

- The automation options available for Dynatrace configuration
- When to use Settings API, Monaco, Terraform, Workflows, or SDKs
- How to choose the right tool based on your use case and team skills
- Best practices for combining automation tools

Key Takeaway: Choose the automation tool that matches your team's skills and your operational model. Monaco is the best starting point for most teams due to its low barrier to entry and GitOps compatibility.

Continue to AUTOM-02: Settings API to learn direct REST API configuration.

This notebook was AI-generated from community-submitted and publicly available sources. This notebook series is not officially supported by Dynatrace. Always verify information against official Dynatrace documentation.